

Full Regression Test Report

Project Title: RentEasy

Student Name: Nuevas, Josh Anton K.

Course / Section: IT342-G1

Date: May 7, 2026

Branch: vertical-slice-regression

1. Project Information

System Description:

RentEasy is a multi-platform equipment rental application. Users can register, log in, browse approved rental products, list rental products, manage cart items, and access protected user/admin areas. Administrators can review pending product listings.

Architecture:

The project uses a Spring Boot backend, a React/Vite web frontend, and a Kotlin Android mobile application. The project was refactored from a technical-layer folder structure into a vertical slice structure grouped by feature.

Testing Scope:

Backend service-layer automated tests were executed with JUnit 5 and Mockito. The web frontend was verified with ESLint and a production Vite build. Android source was refactored into feature packages, but Gradle test execution on this machine was blocked by an installed Java 25 runtime incompatible with the current Gradle/Kotlin setup.

2. Refactoring Summary

The project was refactored from horizontal slicing to vertical slicing.

Before refactoring, backend files were grouped by technical layers:

```
controller/  
service/  
repository/  
model/  
dto/  
config/  
security/
```

After refactoring, backend files are grouped by feature/module:

```
auth/  
users/  
listings/
```

```
cart/  
admin/  
core/
```

The web frontend was refactored from a single shared components folder into feature folders:

```
features/auth/  
features/listings/  
features/cart/  
features/admin/  
shared/  
app/
```

The mobile application was refactored from root-level Kotlin files into feature/core packages:

```
features/auth/  
features/dashboard/  
core/network/
```

All public API routes and user-facing routes were preserved.

3. Updated Project Structure

Backend

```
backend/src/main/java/edu/cit/nuevas/renteasy/  
  BackendApplication.java  
  admin/  
    AdminSeeder.java  
  auth/  
    AuthController.java  
    LoginRequest.java  
    RegisterRequest.java  
    UserService.java  
  cart/  
    CartController.java  
    CartItem.java  
    CartItemRepository.java  
    CartService.java  
  core/  
    config/  
      PasswordConfig.java  
      SecurityConfig.java  
    dto/  
      ApiResponse.java  
    security/  
      JwtAuthenticationFilter.java  
      JwtUtil.java  
  listings/  
    Product.java  
    ProductController.java  
    ProductRepository.java  
    ProductRequest.java  
    ProductService.java  
  users/  
    User.java  
    UserController.java  
    UserRepository.java
```

Web Frontend

```
web/src/  
  app/  
    App.jsx  
  features/
```

```

admin/
  AdminDashboard.jsx
  PendingApproval.jsx
  admin.api.js
auth/
  Login.jsx
  Register.jsx
  auth.api.js
cart/
  Cart.jsx
  cart.api.js
listings/
  CreateListing.jsx
  Home.jsx
  MyListings.jsx
  listings.api.js
shared/
  ProtectedRoute.jsx
  apiClient.js
main.jsx

```

Mobile

```

mobile/app/src/main/java/com/example/it342_mobile_auth/
  core/
    network/
      RetrofitClient.kt
  features/
    auth/
      AuthApi.kt
      AuthModels.kt
      MainActivity.kt
      RegisterActivity.kt
    dashboard/
      DashboardActivity.kt

```

4. Test Plan Documentation

Testing Strategy:

1. Automated backend unit tests validate major service behavior after package movement.
2. Web lint verifies frontend import correctness and React hook correctness.
3. Web production build verifies route/component/module bundling after the feature folder refactor.
4. Android package and manifest paths were updated and statically verified. Gradle test execution requires a compatible JDK.

Functional Requirements Coverage

Feature Module	Requirement Description	Target Platform	Test Case ID
Authentication	User registration, login, JWT generation, protected route access	Backend, Web, Mobile	TC-AUTH-01
Listings	Create product listing with owner, price, stock, category, image, status	Backend, Web	TC-LIST-01
Listings	Browse approved listings	Backend, Web	TC-LIST-02
Admin	Review pending product listing and approve/reject status	Backend, Web	TC-ADMIN-01
Cart	Add product to cart, increment existing item, update quantity, remove item	Backend, Web	TC-CART-01

| User | Access protected user info endpoint | Backend | TC-USER-01 |

| Mobile Auth | Android login/register through Retrofit client | Mobile, Backend | TC-MOB-01 |

Manual Test Scripts

TC-AUTH-01: Authentication

Steps:

1. Open the web login/register pages.
2. Register a user with first name, last name, email, and password.
3. Log in with the registered credentials.
4. Confirm token is stored and the user is redirected to the home page.
5. Attempt access to a protected route without a token.

Expected Result:

The system creates the account, authenticates valid users, rejects invalid credentials, and protects restricted pages.

TC-LIST-01: Create Listing

Steps:

1. Log in as a user.
2. Open Create Listing.
3. Enter name, category, price, stock, description, and image.
4. Submit the listing.

Expected Result:

The listing is saved under the logged-in owner and starts with pending status.

TC-LIST-02: Browse Listings

Steps:

1. Open the home/listings page.
2. Load approved products.
3. Confirm only approved products appear.

Expected Result:

Approved products are displayed with name, price, image, and add-to-cart action.

TC-ADMIN-01: Admin Listing Review

Steps:

1. Log in with an admin email ending in @renteasy.com.
2. Open Admin Dashboard.
3. Select Pending Approval.
4. Approve or reject a pending product.

Expected Result:

The product status updates and the item is removed from the pending queue.

TC-CART-01: Cart Management

Steps:

1. Add a product to cart from the listings page.
2. Open the cart page.
3. Increase and decrease quantity.
4. Remove the cart item.

Expected Result:

Cart items load for the logged-in email, quantity changes persist, and deleted items disappear.

TC-MOB-01: Mobile Authentication

Steps:

1. Open the Android app.
2. Register a new account.
3. Log in with valid credentials.
4. Confirm token is saved to SharedPreferences.
5. Confirm the app navigates to DashboardActivity.

Expected Result:

Retrofit calls the backend auth endpoints and the user is routed to the dashboard after successful login.

Automated Test Cases

| Automated Test | Coverage |

| --- | --- |

| UserServiceTest.registerCreatesUserWhenEmailsAvailable | Registration mapping and password hashing |

UserServiceTest.registerRejectsDuplicateEmail	Duplicate email rejection
UserServiceTest.loginReturnsJwtForValidCredentials	Successful login token flow
UserServiceTest.loginRejectsInvalidCredentials	Invalid password rejection
ProductServiceTest.addProductMapsListingPayloadAndOwner	Listing creation and owner mapping
CartServiceTest.addItemToCartCreatesNewCartItemWhenProductIsNotYetInCart	New cart item creation
CartServiceTest.addItemToCartIncrementsExistingQuantity	Existing cart item increment
CartServiceTest.updateQuantityPersistsChangedQuantity	Cart quantity update
BackendApplicationTests.applicationEntryPointCanBeLoaded	Backend entry point load

5. Regression Test Results

Automated Backend Test Results

Command:

```
mvn.cmd test
```

Result: PASS

Metric Result
--- ---
Total Tests Run 9
Passed 9
Failed 0
Errors 0
Skipped 0

Web Frontend Results

Command:

```
npm.cmd run lint
```

Result: PASS - 0 errors, 0 warnings.

Command:

```
npm.cmd run build
```

Result: PASS - Vite production build completed successfully.

Mobile Results

Command attempted:

```
.\gradlew.bat testDebugUnitTest
```

Result: BLOCKED BY ENVIRONMENT

The command failed before compiling project code because Java 25 is installed locally and the current Gradle/Kotlin setup cannot parse Java version 25. The project should be verified with JDK 17, which is the expected Java version for this Android Gradle setup.

6. Issues Found and Fixes Applied

Issue 1: Backend package imports broke after file movement.

Root Cause:

Controllers, services, models, repositories, DTOs, config, and security classes were moved from layer packages into feature packages.

Fix Applied:

Updated Java package declarations and imports across auth, users, listings, cart, admin, and core packages. Backend tests passed after the fix.

Issue 2: Web direct fetch calls were scattered across screens.

Root Cause:

Network behavior was embedded directly in React components, making the web app less aligned with vertical slicing.

Fix Applied:

Added feature API modules: auth.api.js, listings.api.js, cart.api.js, and admin.api.js. Components now call their slice API modules.

Issue 3: React hook dependency warning in PendingApproval.

Root Cause:

The pending listing fetch function was referenced from useEffect without a stable dependency.

Fix Applied:

Wrapped the function with useCallback and added the dependency to useEffect. ESLint now passes cleanly.

Issue 4: Android Gradle verification blocked by Java 25.

Root Cause:

Gradle/Kotlin tooling failed while parsing the installed Java version before project code compilation.

Fix Applied:

No source-code fix was required. The required execution fix is to run Android verification using JDK 17 or upgrade Gradle/Kotlin/AGP to versions compatible with Java 25.

7. Automated Test Evidence

Evidence file:

`docs/AutomatedTestEvidence_RentEasy.md`

Generated backend test reports:

backend/target/surefire-reports/

Backend command evidence:

Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Web command evidence:

npm.cmd run lint: PASS
npm.cmd run build: PASS

8. Submission Checklist

Requirement	Status
-------------	--------

---	---
-----	-----

Refactor branch created	Done
-------------------------	------

Backend vertical slice refactor	Done
---------------------------------	------

Web vertical slice refactor	Done
-----------------------------	------

Mobile vertical slice refactor	Done
--------------------------------	------

Backend automated tests	Done
-------------------------	------

Web lint/build verification	Done
-----------------------------	------

Mobile Gradle test attempt documented	Done, blocked by Java 25
---------------------------------------	--------------------------

Full regression report	Done
------------------------	------